

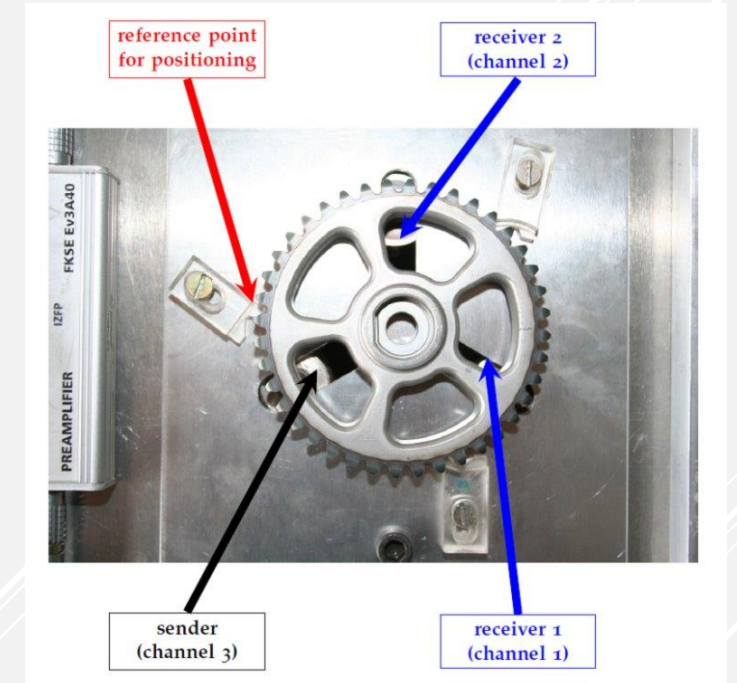
# EXPLORATIVE ANALYSE DER ZFP-DATEN VON ZAHNRÄDERN

Datum: 19. 06. 2025

Modul: Künstliche in der Materialdiagnostik

Dozenten: Prof. Dr.-Ing. Constanze Tschöpe, Dr. Ivan Kraljevski

Gruppenmitglieder: Talib Gasanov, Dmitriy Kozyrev, Eric Sprunk, Volodymyr Telesh



# INHALT

1. Einführung und Aufgabenstellung
2. Datensatz und Laden der Daten
3. Merkmalsanalyse der Signale
4. Principal Component Analysis (PCA) für die Reduzierung der Dimensionen
5. Visualisierung und Evaluation

# 1. EINFÜHRUNG UND AUFGABENSTELLUNG

Explorative Analyse ist entscheidend, um Muster und Trends in Daten zu finden sowie Ausreißer zu identifizieren.

Zerstörungsfreie Prüfverfahren sind wichtig, um Zahnräder (oder andere Materialien) intakt zu halten.

## **Aufgabe:**

- Lade die gegebenen Signale in einen DataFrame.
- Wende die Hauptkomponentenanalyse (PCA) an, um die Dimensionalität der Merkmale zu reduzieren.
- Visualisiere die Datenpunkte in 2D-Diagrammen.
- Werte die Ergebnisse aus.

## 2. DATEN

- Datensatz: 5 Zahnräder (Z01–Z05)
- 110 Positionsdateien (Z01: 50, Z04: 30, andere: 10 jedes)
- Jede Positionsdatei: 2 Kanäle mit je 2 Varianten
- Gesamt: 440 .wav-Dateien

## 2. DATEN

- Wir nutzen `os.path.join` für das Lesen der Daten
- Wir holen uns alle Daten mit `.wav` am Ende

```
# - Durchsucht Unterordner Z01 bis Z05 jeweils in deren Wav-Unterordner
pattern = os.path.join('*Datei', 'sig', 'Z*', 'Z*', 'Z*', 'Wav', '*.wav')

# Alle Dateien finden, die dem Muster entsprechen
files = glob.glob(pattern)
print(len(files), 'files', '\n', files[264]) # Print for Error-Checks
```

- In 'entry' setzen wir die Schlüssel gemäß der Aufgabenstellung

- Danach fügen wir dies der Datenliste hinzu

```
# Extrahiert Dateinamen
basename = os.path.basename(fn)
parts = basename.split('_')

# Erstellt Datensatz mit Metadaten und Signal (Keys der Aufgabenstellung entsprechend)
entry = {
    'fn': basename,
    'sig': signal,
    'spec': parts[0],
    'mID': parts[4],
    'time': parts[5],
    'rID': parts[6],
    'sID': parts[8][:4],
    'stft': None
}

data.append(entry) # Fügt Datensatz zur Liste hinzu

# Erstellt DataFrame aus der Liste
df = pd.DataFrame(data)

# Zeigt die ersten 5 Einträge zur Qualitätskontrolle (Error-Checking)
print("DataFrame-Übersicht:")
```

### 3. MERKMALSANALYSE DER SIGNALE

Wir extrahieren die folgenden Merkmale:

- **Betragsmittelwert:** Mittelwert des Betrags über die Zeit gemittelt
- **Spektraler Mittelwert:** Durchschnittliche Frequenz, gewichtet nach Betrag
- **Spektrale Varianz:** Abstand jedes Frequenzbereichs vom Mittelwert
- **Obere Entropie:** Beschreibt, wie unregelmäßig der obere Spektralbereich ist
- **Flatness (Spektrale Flachheit):** Zeigt an, wie viel Rauschen im Signal enthalten ist
- **Rolloff-Frequenz:** 85 % der Energie liegen unterhalb dieser Frequenz
- **Obere Kurtosis:** Gibt die Spitzigkeit oder Asymmetrie im oberen Bereich an

### 3. MERKMALSANALYSE DER SIGNALE

Mehr Signalmerkmale:

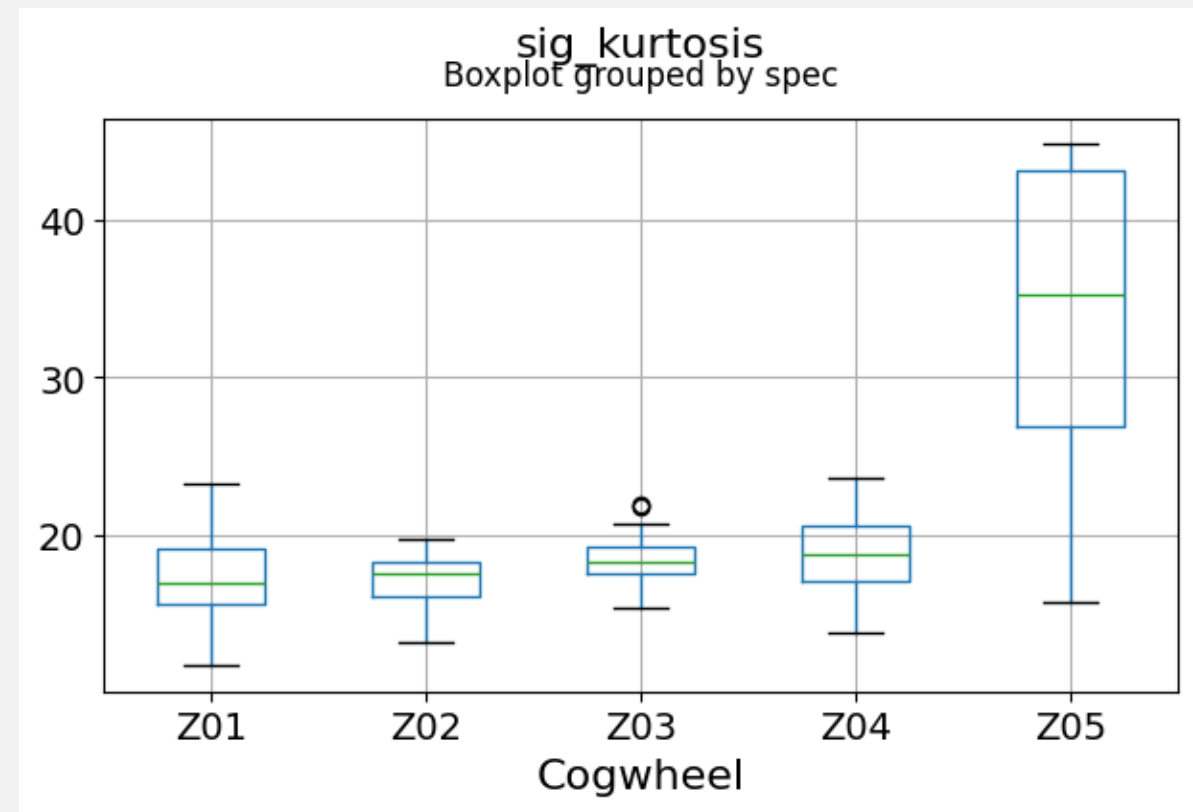
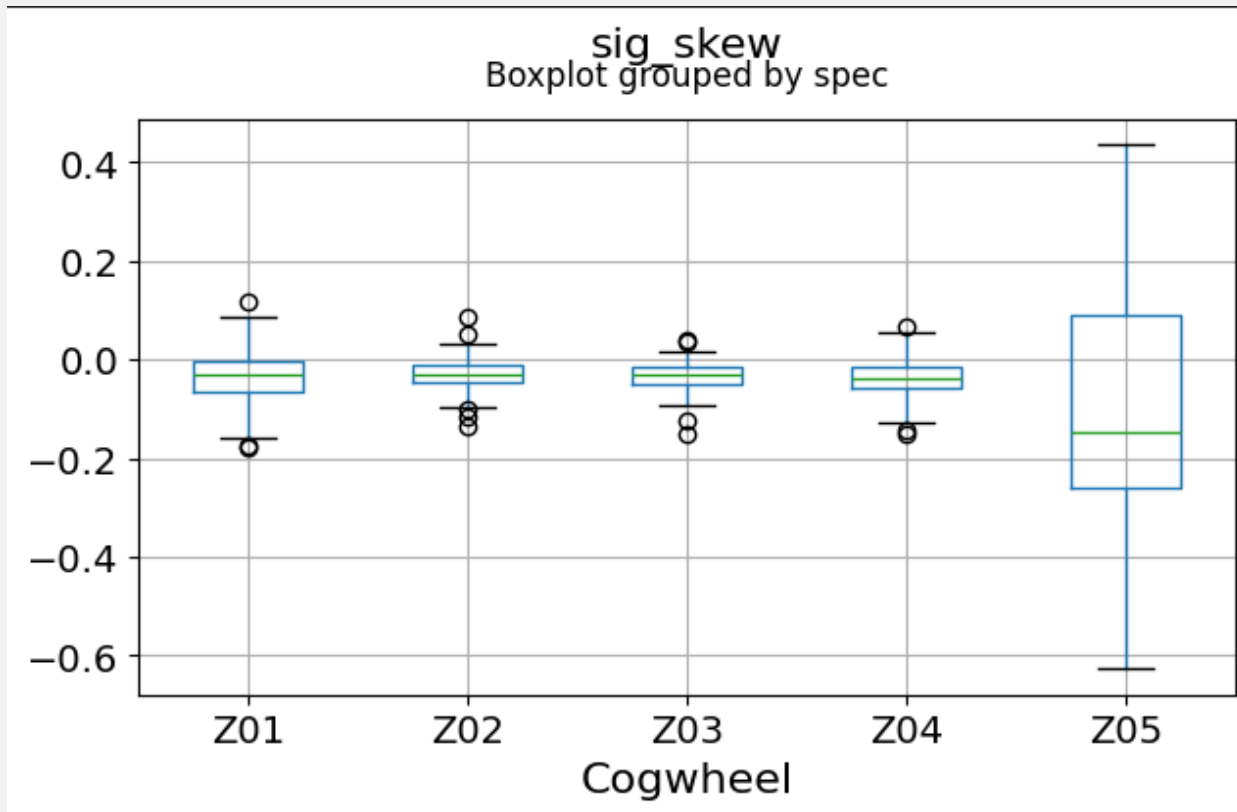
```
def analyze_signal(sig):  
    signal_mean = np.mean(sig)  
    signal_variance = np.var(sig)  
    signal_skewness = skew(sig)  
    signal_kurtosis = kurtosis(sig) # fourth moment  
    signal_rms = np.sqrt(np.mean(sig**2))  
    signal_peak = np.max(np.abs(sig)) # peak amplitude  
    return signal_mean, signal_variance, signal_skewness, signal_rms, signal_peak, signal_kurtosis
```

Die Merkmalsliste:

```
featureslist = ['sig_mean', 'sig_var', 'sig_skew', 'sig_rms', 'sig_peak',  
                'sig_kurtosis', 'spec_mean', 'spec_var', 'spec_entropy_up', 'spec_flatness',  
                'spec_rolloff_freq', 'spec_kurtosis'] # added new features
```

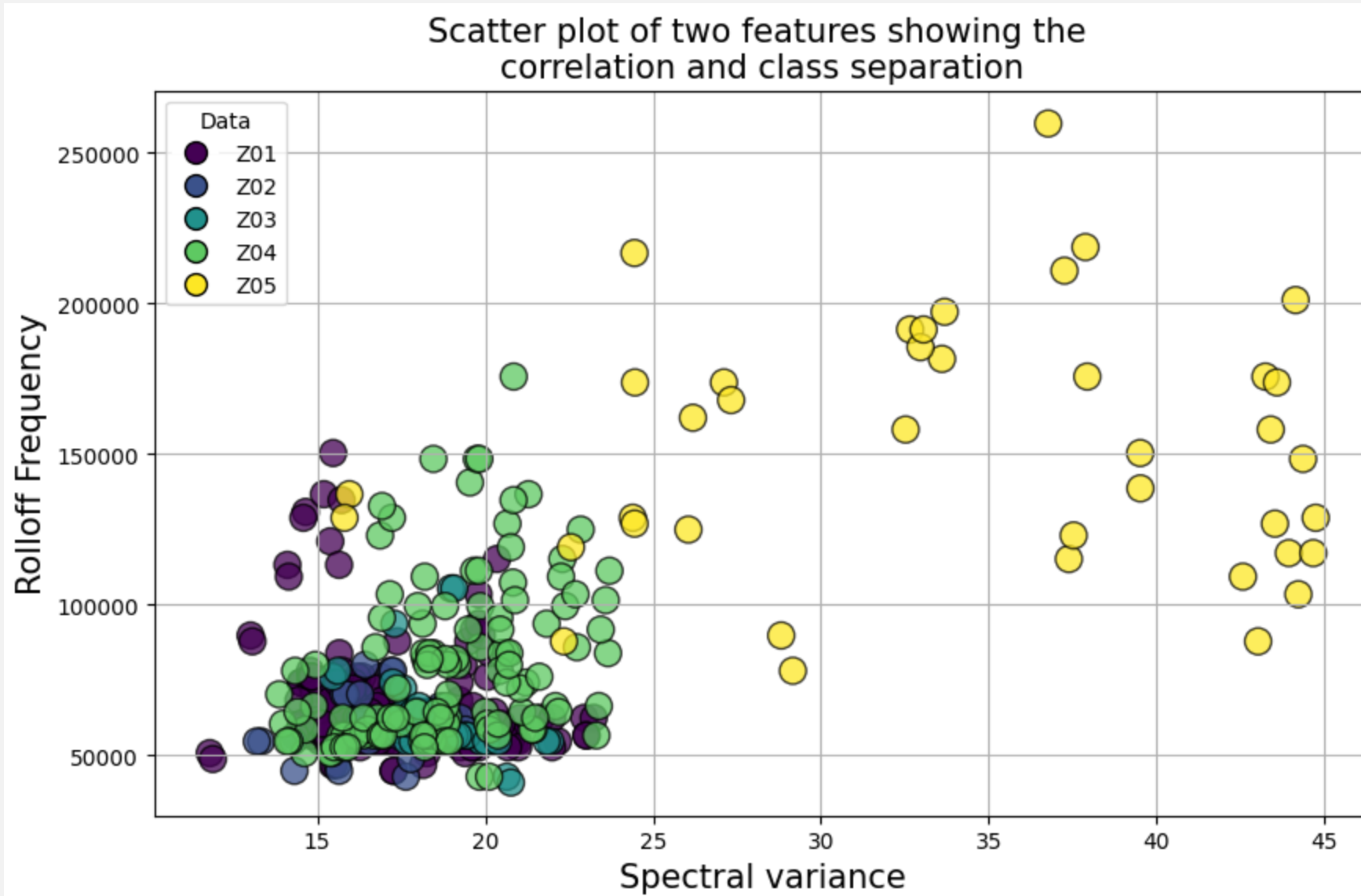
### 3. MERKMALSANALYSE DER SIGNALE

Boxplots für das Verzerren und die Kurtosis:



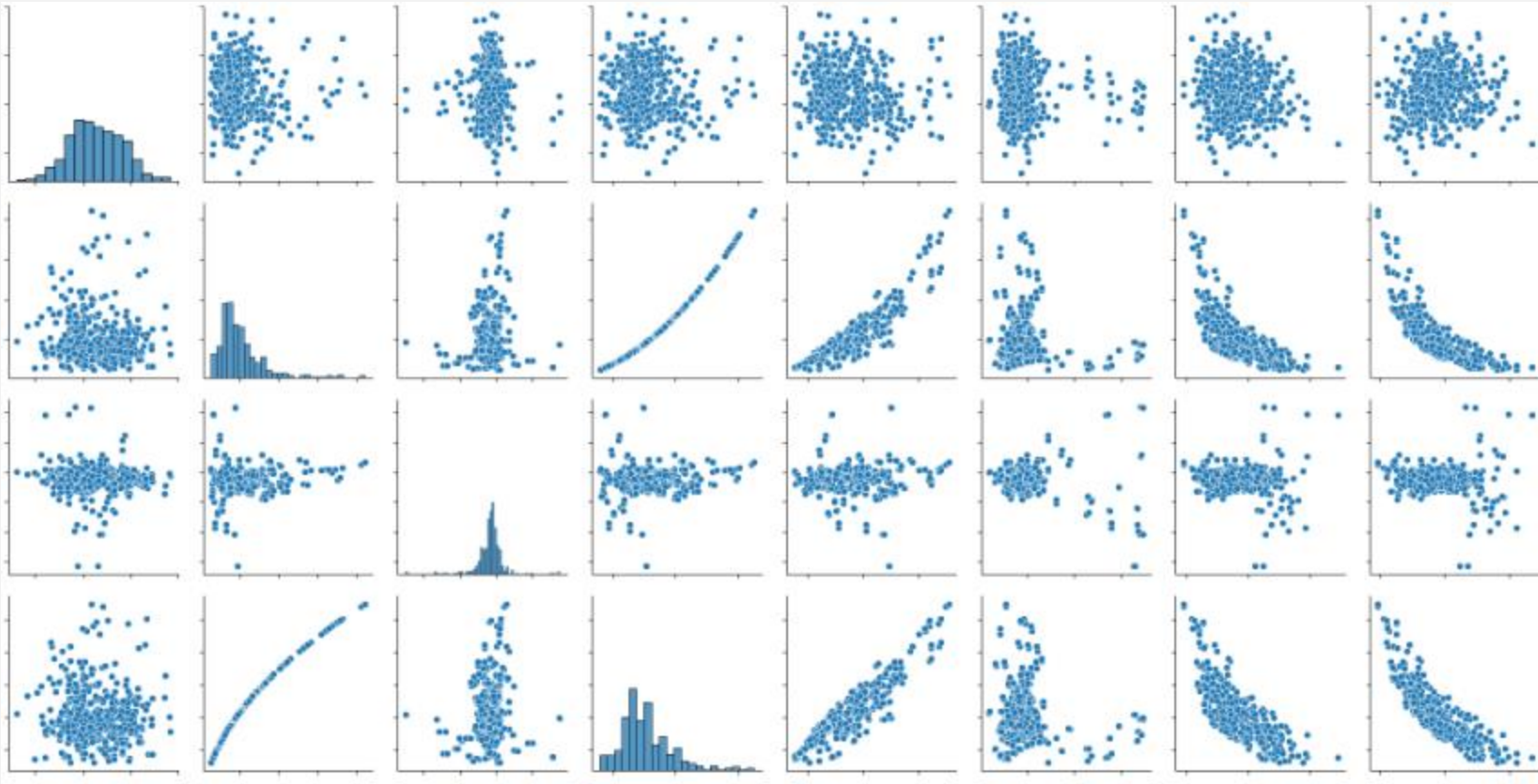


### 3. MERKMALSANALYSE DER SIGNALE



### 3. MERKMALSANALYSE DER SIGNALE

Merkmalsübersicht mit Seaborn:



## 4. PRINCIPAL COMPONENT ANALYSIS

```
pca = PCA(n_components=None)  
dfx_pca = pca.fit(dfx)
```

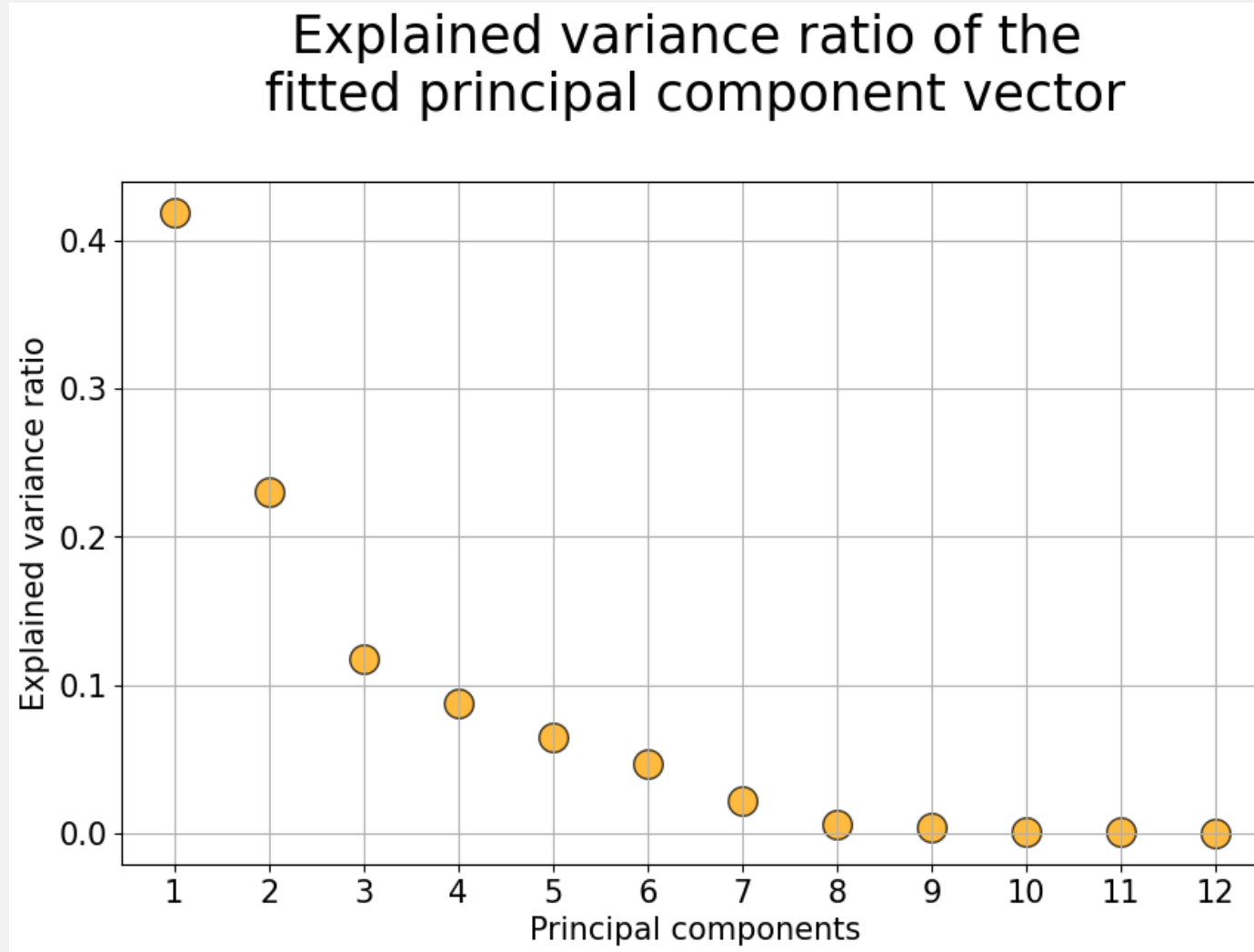
### **Ziel der PCA:**

Reduktion der Anzahl an Variablen durch die Verwendung neuer, unkorrelierter Komponenten, die möglichst viel Varianz der Originaldaten erklären.

PCA berechnet lineare Kombinationen der ursprünglichen Variablen, wobei die erste Hauptkomponente die größte Varianz erfasst.

Das erklärte Varianzverhältnis zeigt, wie viel der Gesamtvarianz durch jede Hauptkomponente erklärt wird.

## 4. PRINCIPLE COMPONENT ANALYSIS



## 4. REDUKTION DER DIMENSION

### Ziel der Dimensionsreduktion:

Die Anzahl der Variablen in einem Datensatz reduzieren, ohne wichtige Informationen zu verlieren

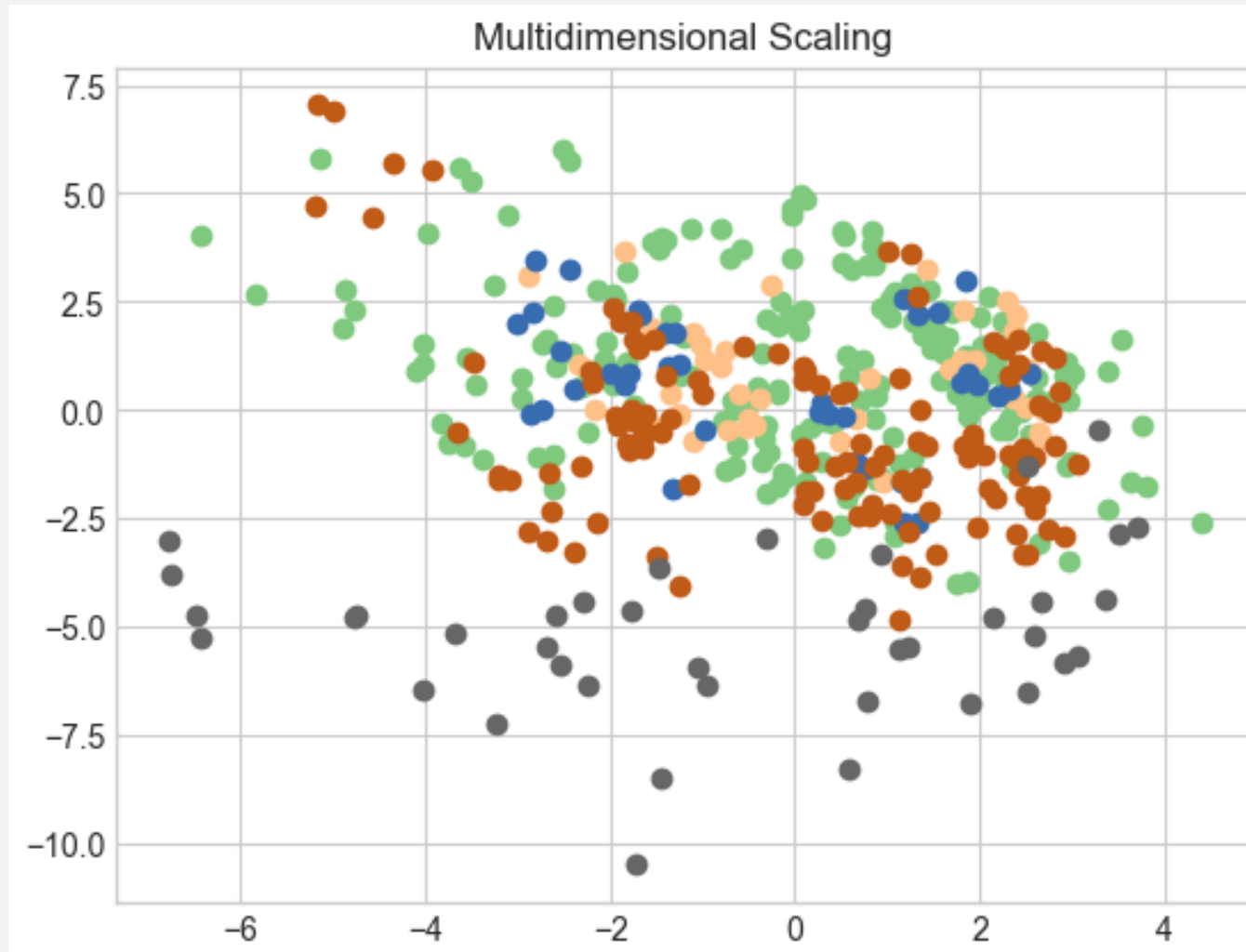
Wir erstellen 2D- und 3D-Scatterplots.

Dimensionalität reduzieren:

```
do_plot(PCA(n_components=2).fit_transform(dfx), 'PCA', df['spec'].values)
```

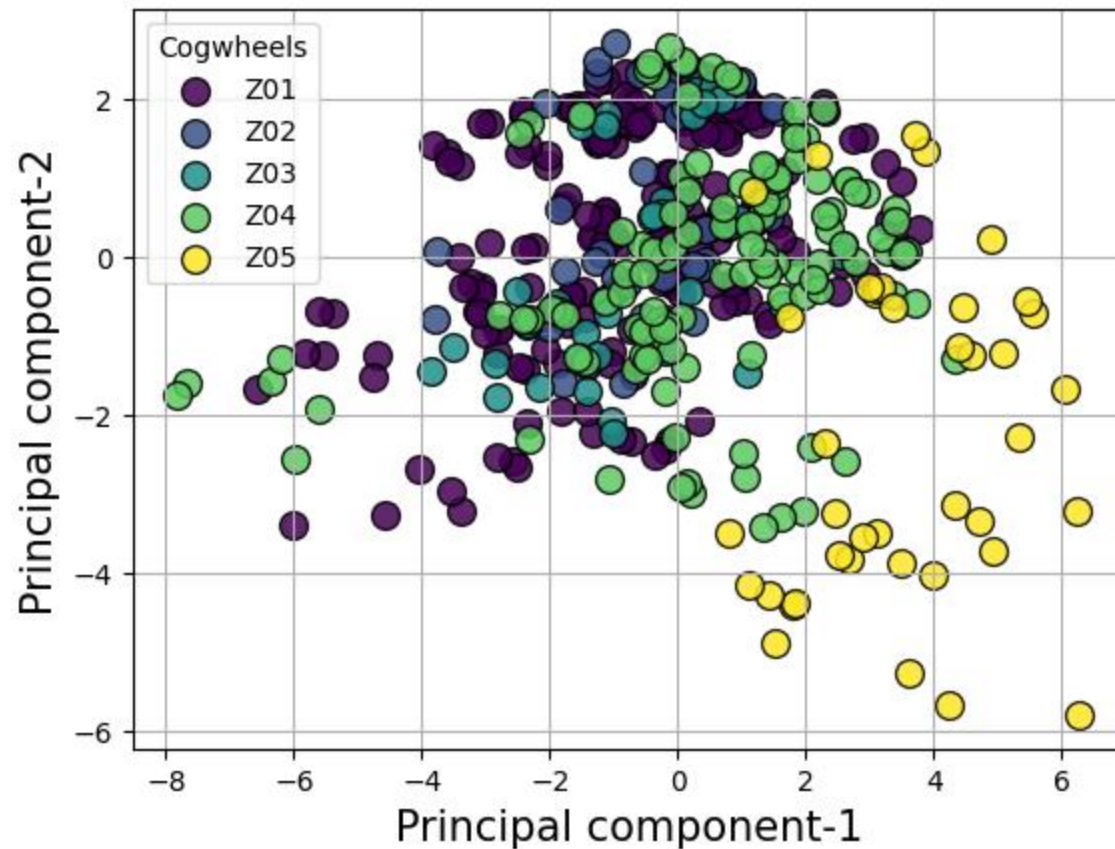
```
with plt.style.context(plt_style):
    fig = plt.figure()
    if dimension == 2:
        ax = fig.add_subplot(111)
        for lab, col in zip(label_types, colors):
            ax.scatter(X_fit[labels==lab, 0],
                      X_fit[labels==lab, 1],
                      color=col)
    elif dimension == 3:
        ax = fig.add_subplot(111, projection='3d')
        for lab, col in zip(label_types, colors):
            ax.scatter(X_fit[labels==lab, 0],
                      X_fit[labels==lab, 1],
                      X_fit[labels==lab, 2],
                      color=col)
```

## 5. VISUALISIERUNG DER DATENPUNKTE



## 5. VISUALISIERUNG DER DATENPUNKTE

Class separation using first two principal components



## 5. EVALUATION

Wir sehen, dass die Datenpunkte im Diagramm um den Punkt  $(0,0)$  liegen.

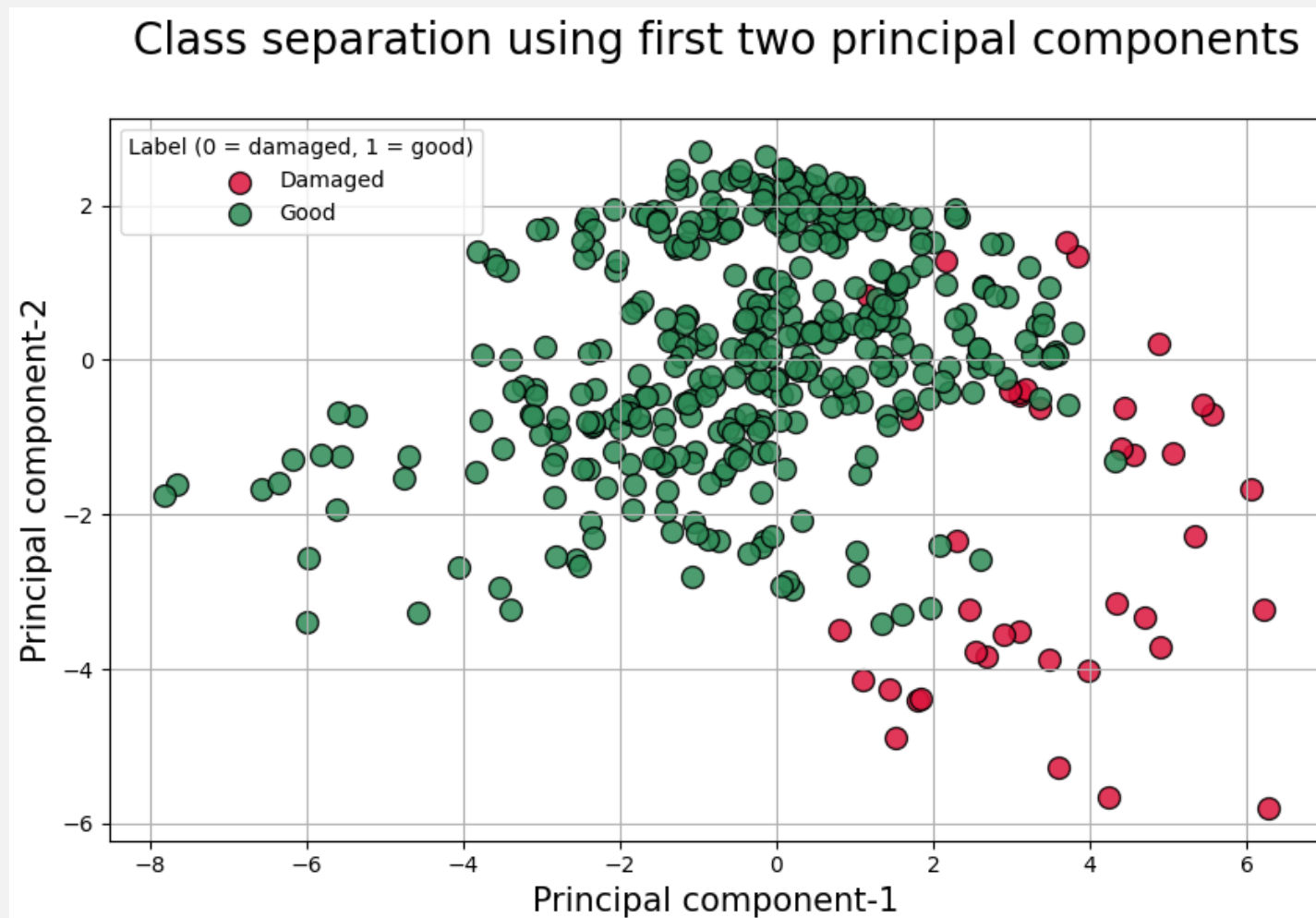
Z01, Z02, Z03 und Z04 ähneln sich, da die meisten ihrer Datenpunkte nahe bei  $(0,0)$  liegen.

Z05 ist im Vergleich zu Z01 – Z04 etwas verschoben.



## 5. EVALUATION

Als Ergebnis erhalten wir folgendes Separierungsdiagramm:



## 5. EVALUATION

Wir sehen, dass die Datenpunkte von Z01 – Z04 als OK erscheinen.

Die Datenpunkte von Z05 hingegen wirken auffällig.

Wir schließen daraus:

Z05 ist ein beschädigtes Zahnrad, während alle anderen Zahnräder in gutem Zustand sind.

Aber kein Zahnrad ist perfekt! Wenn eines perfekt wäre, lägen alle seine Datenpunkte bei (0,0).

DANKE FÜR IHRE AUFMERKSAMKEIT